

Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Дисциплина «Информационная безопасность»

Работа 1:
Разработка защищенного REST API
с интеграцией в CI/CD

Группа: Р3408

Выполнил: Таджеддинов Р. Э.

ССЫЛКА НА РЕПОЗИТОРИЙ

Код проекта размещён в публичном репозитории GitHub:
<https://github.com/r4m63/infobez-itmo>.

ТЕКСТ ФАЙЛА README.MD

REST API на Java 25 + Spring Boot 4.1.1 (Maven). Данные хранятся в H2 в памяти, доступ к БД через Hibernate (JPA). Реализована защита от SQL-инъекций и XSS, аутентификация по JWT с хэшированием паролей bcrypt. При каждом push и pull_request GitHub Actions запускает SAST (SpotBugs) и SCA (OWASP Dependency-Check).

- Репозиторий: <https://github.com/r4m63/infobez-itmo>
- Последний успешный запуск pipeline: <https://github.com/r4m63/infobez-itmo/actions/runs/36620490824/job/109584335296>

API

Запуск: ./mvnw spring-boot:run, приложение слушает <http://localhost:8080>.
Учебные пользователи: admin / admin-password и user / user-password.

Таблица 1 — API

Метод	Путь	Доступ	Назначение
POST	/auth/login	без токена	Проверяет логин и пароль, возвращает JWT
GET	/api/data	только аутентифицированные	Список постов; ?query= ищет по заголовку
POST	/api/posts	только аутентифицированные	Создаёт пост

POST /auth/login

```
curl -X POST localhost:8080/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"admin-password"}'
{"accessToken":"eyJhbGciOiJIUzI1NiJ9...", "tokenType":"Bearer", "expiresIn":900}
```

Неверный логин или пароль: 401.

GET /api/data

```
curl localhost:8080/api/data -H "Authorization: Bearer $TOKEN"
```

```
{"items":[{"id":2,"title":"Второй  
пост","content":"...", "author":"user", "createdAt":"...",  
...], "total":2}
```

Без токена или с недействительным токеном: 401.

POST /api/posts

```
curl -X POST localhost:8080/api/posts \  
-H "Authorization: Bearer $TOKEN" -H 'Content-Type:  
application/json' \  
-d '{"title":"Заголовок", "content":"Текст"}'
```

Ответ 201 Created с созданным постом. Без токена: 401.

Меры защиты

Защита от SQL-инъекций

SQL-запросы не собираются конкатенацией строк: все обращения к БД идут через ORM Hibernate (Spring Data JPA). Поиск по заголовку использует JPQL с именованным параметром ([PostRepository.java](#)):

```
@Query("SELECT p FROM Post p WHERE LOWER(p.title) LIKE  
LOWER(CONCAT('%', :query, '%')) ORDER BY p.createdAt DESC")  
List<Post> searchByTitle(@Param("query") String query);
```

Hibernate выполняет его как prepared statement: значение :query передаётся в БД отдельно от текста запроса. Нагрузка ' OR '1'='1 ищется как обычная строка и возвращает пустой список.

Защита от XSS

Все пользовательские данные в ответах API экранируются встроенной функцией Spring `HtmlUtils.htmlEscape` ([ApiController.java](#)):

```
private static PostDto escape(Post post) {  
    return new PostDto(post.getId(),  
        HtmlUtils.htmlEscape(post.getTitle()),  
        HtmlUtils.htmlEscape(post.getContent()),  
        HtmlUtils.htmlEscape(post.getAuthor()), post.getCreatedAt());  
}
```

Заголовок	<code><script>alert(1)</script></code>	возвращается	как
	<code>&lt;script&gt;alert(1)&lt;/script&gt;</code>		

Аутентификация

- Хэширование паролей. Пароли не хранятся в открытом виде: в `application.yaml` записаны только bcrypt-хэши (`BCryptPasswordEncoder`, `cost 12`). При входе `passwords.matches(...)` сравнивает введенный пароль с хэшем.
- Выдача JWT. При успешном входе `/auth/login` возвращает JWT, подписанный HS256, с полями `iss`, `sub` (логин), `iat`, `exp`. Срок жизни токена 15 минут.
- Middleware проверки JWT. В [SecurityConfig.java](#) фильтр Spring Security (OAuth2 Resource Server) проверяет токен из заголовка `Authorization: Bearer` на всех защищённых эндпоинтах: подпись, срок действия и издателя. Без токена открыт только `/auth/login`, на остальные запросы без валидного токена возвращается 401.

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/auth/login").permitAll()
    .anyRequest().authenticated())
.oauth2ResourceServer(oauth -> oauth.jwt(jwt -> jwt.decoder(decoder)))
```

Отчёты SAST/SCA

Pipeline [.github/workflows/ci.yml](#) запускается при каждом `push` и `pull_request`. SAST выполняет SpotBugs с плагином Find Security Bugs, SCA выполняет OWASP Dependency-Check. Скриншоты относятся к [запуску #54](#).

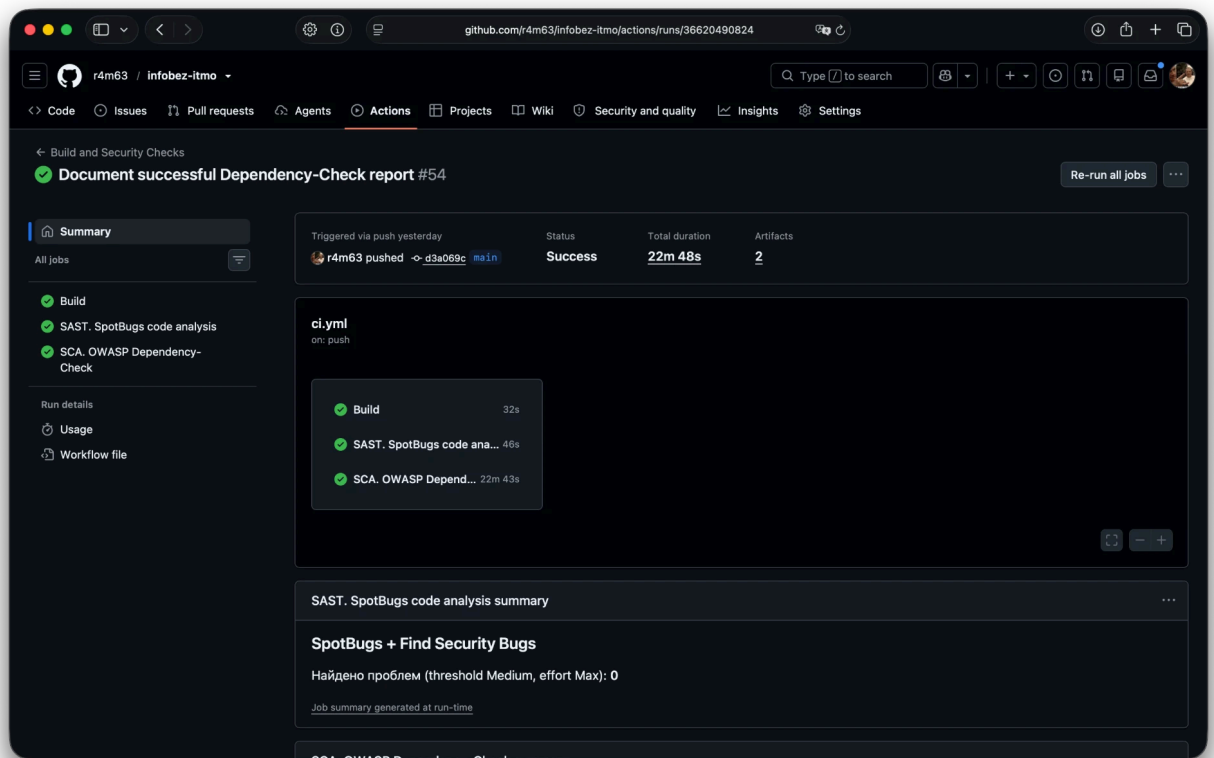


Рисунок 1 — Успешный запуск GitHub Actions

Успешный запуск pipeline, в Job Summary SpotBugs найдено 0 проблем (рисунок 1).

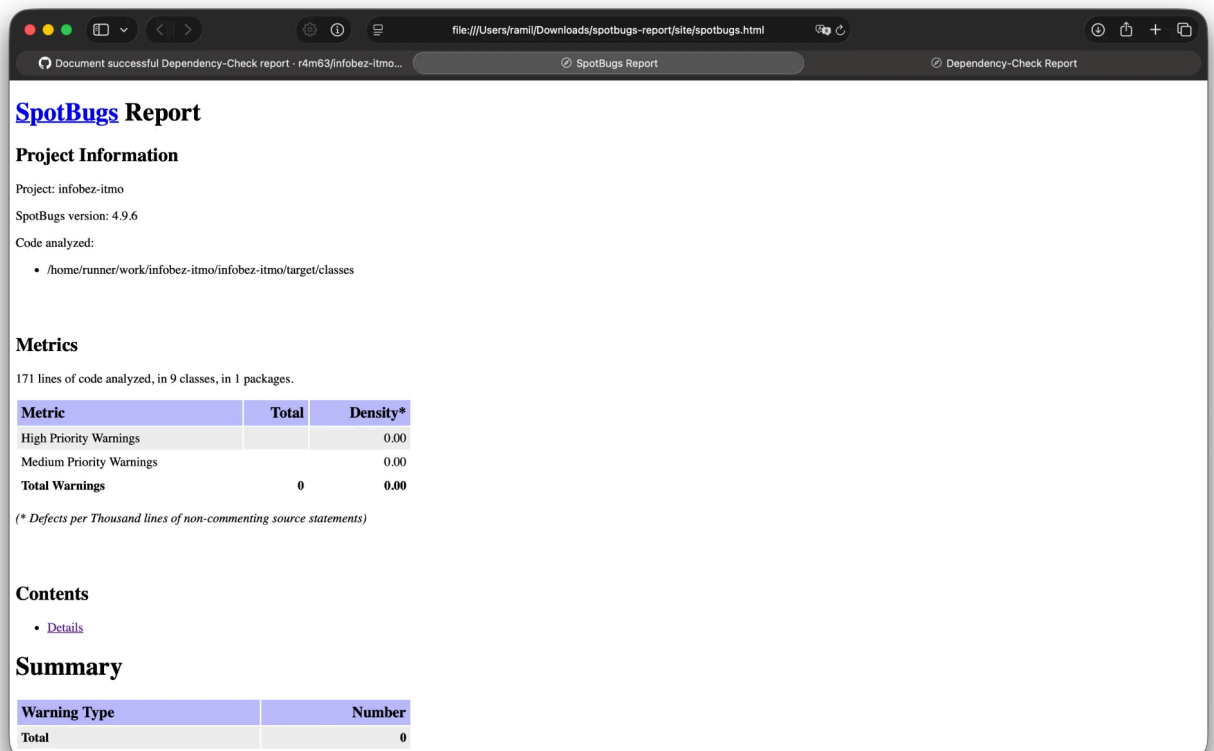


Рисунок 2 — Отчёт SpotBugs

SAST, SpotBugs + Find Security Bugs: 0 замечаний (рисунок 2).

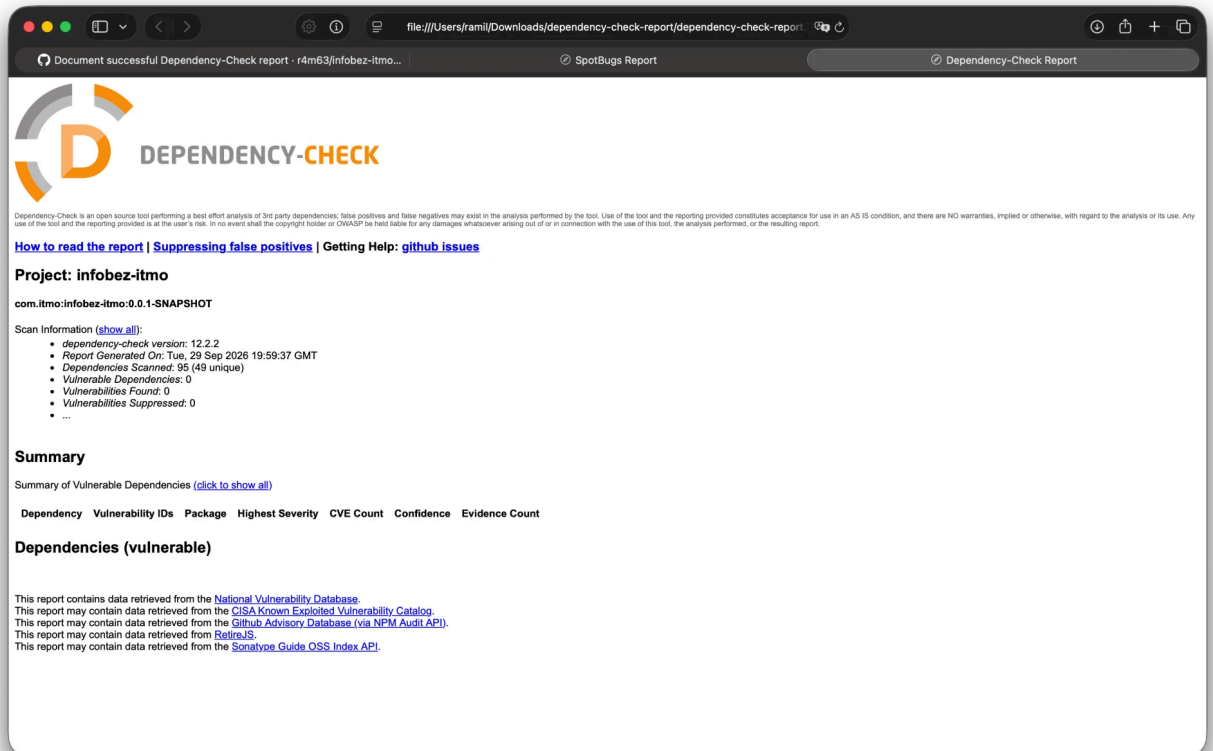


Рисунок 3 — Отчёт OWASP Dependency-Check

SCA, OWASP Dependency-Check: 95 зависимостей (49 уникальных), 0 уязвимостей (рисунок 3).

ССЫЛКА НА ПОСЛЕДНИЙ УСПЕШНЫЙ ЗАПУСК PIPELINE

Запуск #54 workflow Build and Security Checks:

<https://github.com/r4m63/infobez-itmo/actions/runs/36620490824/job/109584335296>.